

POINTERS

Pointers

□ Pointers

- Powerful
- Simulate pass-by-reference
- Close relationship with arrays and strings

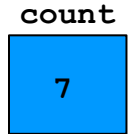
Pointers

- It provides a way of accessing a variable without referring to its name.
- The mechanism used for this is the *address* of the variable.

Pointer Variable Declarations and Initialization

□ Pointer variables

- Contain memory addresses as values
- Normally, variable contains specific value (direct reference)
- Pointers contain address of variable that has specific value (indirect reference)



□ Indirection

- Referencing value through pointer

□ Pointer declarations

- ***** indicates variable is pointer

```
int *myPtr;
```

declares pointer to **int**, pointer of type **int ***

- Multiple pointers require multiple asterisks

```
int *myPtr1, *myPtr2;
```

Pointer Variable Declarations and Initialization

- Can declare pointers to any data type
- Pointer initialization
 - Initialized to **0**, **NULL**, or address
 - **0** or **NULL** points to nothing

Address of variable

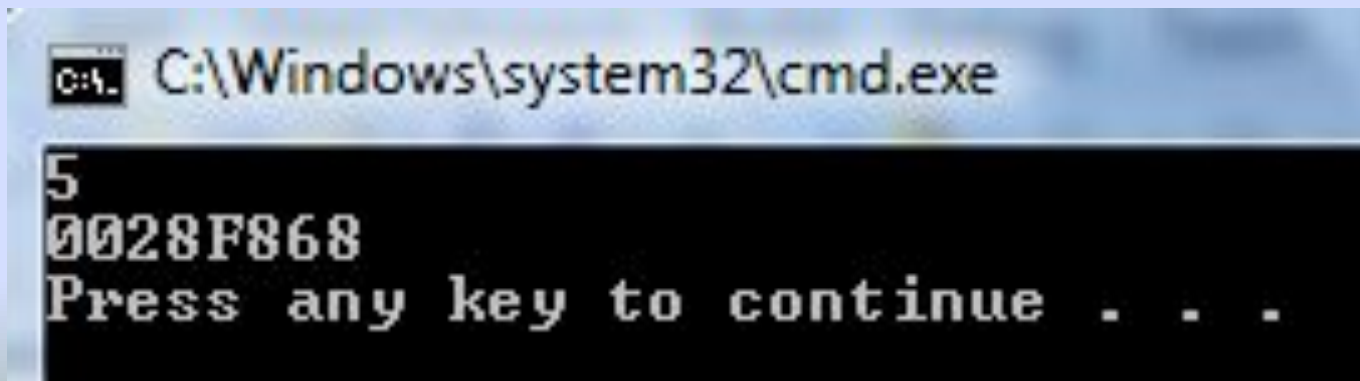
- Each variable occupies some bytes in memory (according to its size)
- Each byte of memory has a unique address so that it can be accessed (just like the address of our homes)
- Variable names are actually titles given to these addresses
 - Such as the 'Wapda house'
- When we use a variable, the compiler fetches the value placed at its address or writes a new value to that address

Address of variable

- How can we find the address of that variable?
- The “&” (Ampersand) operator returns the address of a variable

```
cout << a << endl;
```

```
cout << &a << endl;
```



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\system32\cmd.exe'. The command prompt displays the output of a program: the number '5' on the first line, the memory address '0028F868' on the second line, and the text 'Press any key to continue . . .' on the third line.

```
C:\Windows\system32\cmd.exe  
5  
0028F868  
Press any key to continue . . .
```

Pointers

- Pointers are just variables storing numbers –but those numbers are memory addresses
- A pointer that stores the address of some variable *x* is said to *point to x*.
- A pointer is declared by putting a star (or '*') before the variable name.
- To access the **value** of *x*, we'd need to *dereference* the pointer.

Motivation for using Pointers

- To return more than one value from a function.
- To pass arrays and strings more conveniently from one function to another.
- To manipulate arrays more easily by moving pointers to them.
- Obtaining memory from the system
- In more advanced programming, such as
 - To create complex data structures, such as linked lists and binary trees
 - For dynamic memory allocation and de-allocation

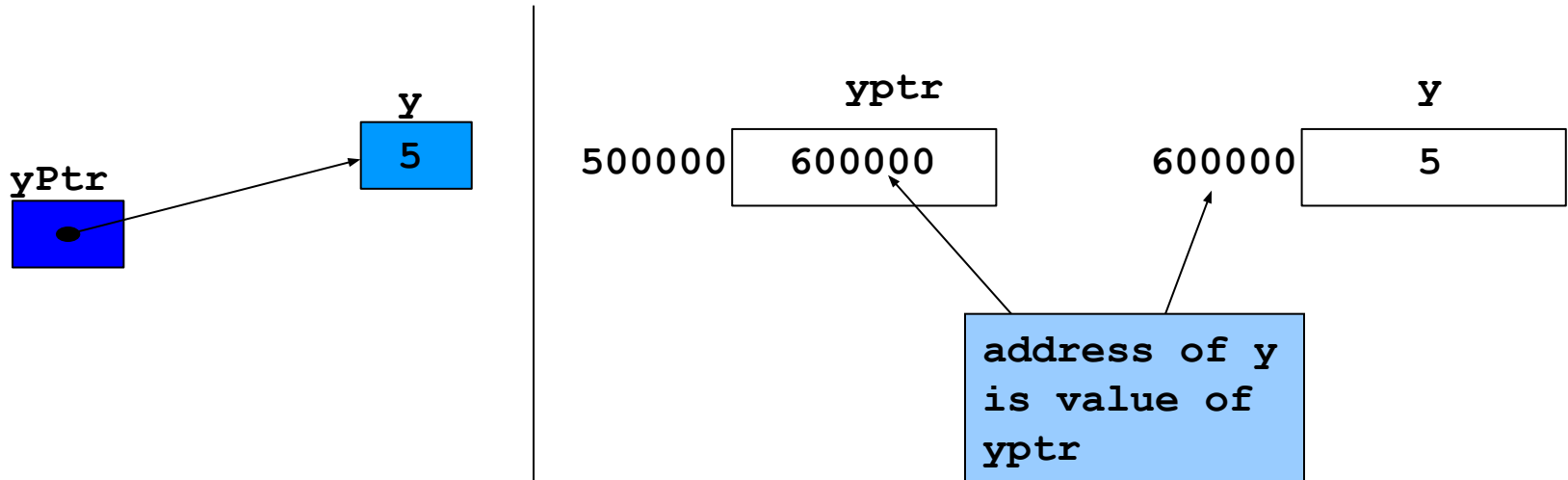
Pointer Operators

□ & (address operator)

- Returns memory address of its operand
- Example

```
int y = 5;  
int *yPtr;  
yPtr = &y;    // yPtr gets address of y
```

- **yPtr** “points to” **y**



Pointer Operators

- ***** (indirection/dereferencing operator)
 - Returns synonym for object its pointer operand points to
 - ***yPtr** returns **y** (because **yPtr** points to **y**).

`*yPtr = 9;            // assigns 9 to y`

- ***** and **&** are inverses of each other

```

1  // Fig. 5.4: fig05_04.cpp
2  // Using the & and * operators.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  int main()
9  {
10     int a;      // a is an integer
11     int *aPtr;  // aPtr is a pointer to an integer
12
13     a = 7;
14     aPtr = &a;  // aPtr assigned address of a
15
16     cout << "The address of a is " << &a
17           << "\nThe value of aPtr is " << aPtr;
18
19     cout << "\n\nThe value of a is " << a
20           << "\nThe value of *aPtr is " << *aPtr;
21
22     cout << "\n\nShowing that * and & are inverses of "
23           << "each other.\n&*aPtr = " << &*aPtr
24           << "\n*&aPtr = " << *&aPtr << endl;
25

```

* and & are inverses
of each other

```
26     return 0; // indicates successful termination
27
28 } // end main
```

The address of a is 0012FED4

The value of aPtr is 0012FED4

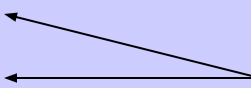
The value of a is 7

The value of *aPtr is 7

Showing that * and & are inverses of each other.

&*aPtr = 0012FED4

*&aPtr = 0012FED4



*** and & are inverses; same
result when both applied to
aPtr**

Calling Functions by Reference

- 3 ways to pass arguments to function
 - Pass-by-value
 - Pass-by-reference with reference arguments
 - Pass-by-reference with pointer arguments
- `return` can return one value from function
- Arguments passed to function using reference arguments
 - Modify original values of arguments
 - More than one value “returned”

Calling Functions by Reference

- Pass-by-reference with pointer arguments
 - Simulate pass-by-reference
 - Use pointers and indirection operator
 - Pass address of argument using **&** operator
 - Arrays not passed with **&** because array name already pointer

```
1 // Fig. 5.6: fig05_06.cpp
2 // Cube a variable using pass-by-value.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 int cubeByValue( int );    // prototype
9
10 int main()
11 {
12     int number = 5;
13
14     cout << "The original value of number is "
15
16     // pass number by value to cubeByValue
17     number = cubeByValue( number );
18
19     cout << "\nThe new value of number is " << number << endl;
20
21     return 0;    // indicates successful termination
22
23 } // end main
24
```

Pass number by value; result
returned by
cubeByValue


```
25 // calculate and return cube of integer argument
26 int cubeByValue( int n )
27 {
28     return n * n * n; // cube local variable
29
30 } // end function cubeByValue
```

cubeByValue receives
parameter passed-by-value

The original value of number is 5
The new value of number is 125

Cubes and **returns**
local variable **n**

```

1 // Fig. 5.7: fig05_07.cpp
2 // Cube a variable using pass-by-reference
3 // with a pointer argument.
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 void cubeByReference( int * );    // prototype
10
11 int main()
12 {
13     int number = 5;
14
15     cout << "The original value of number is " << number << endl;
16
17     // pass address of number to cubeByReference
18     cubeByReference( &number );
19
20     cout << "\nThe new value of number is " << number << endl;
21
22     return 0;    // indicates successful termination
23
24 } // end main
25

```

Prototype indicates parameter
is pointer to **int**

Apply address operator **&** to
pass address of number to
cubeByReference

cubeByReference
modified variable
number

```
26 // calculate cube of *nPtr; modifies variable number in main
27 void cubeByReference( int *nPtr )
28 {
29     *nPtr = *nPtr * *nPtr * *nPtr; // cube
30
31 } // end function cubeByReference
```

cubeByReference
receives address of **int**
variable,
i.e., pointer to an **int**

The original value of number is 5
The new value of number is 125

Modify and access **int**
variable using indirection
operator *****